

Státnicové okruhy – Databáze

Aplikovaná informatika

Obsah

1	Databáze	2
1.1	Analýza, návrh a realizace databáze	3
1.1.1	Databázový systém a jeho vlastnosti	3
1.1.2	Metodologie analýzy a návrhu databáze – P3A	3
1.1.3	Návrh databáze	5
1.1.4	Normalizace databáze	6
1.1.5	Integrita a transakce	7
1.1.6	Indexy	7
1.2	Datové modelování	8
1.2.1	Základní pojmy	8
1.2.2	ER (Entity Relationship) modelování	9
1.2.3	Datové modelování – činnosti	10
1.2.4	Transformace do relačního modelu dat	10
1.2.5	Modely databázových systémů	10
1.2.6	Trendy v databázových systémech	11
1.3	Databázové jazyky	12
1.3.1	Dělení databázových jazyků	12
1.3.2	Databázové jazyky 4. generace (4GL)	13
1.3.3	SQL (Structured Query Language)	13
1.3.4	PL/SQL	14
1.3.5	OQL (Object Query Language)	15
1.3.6	JPQL (Java Persistence Query Language)	15
1.3.7	XQuery	15
1.3.8	SQL injection – bezpečnostní hrozba	15

1 Databáze

Obecné pojmy

Databáze je organizovaný systém souborů pro ukládání dat. Tyto soubory jsou mezi sebou navzájem propojeny. V širším smyslu jsou součástí databáze i softwarové prostředky, které umožňují manipulaci s uloženými daty a přístup k nim. Tento software se v české odborné literatuře nazývá **systém řízení báze dat (SŘBD)**. Běžně se označením databáze myslí jak uložená data, tak i software.

- **data** – formalizované a fyzicky zaznamenané znalosti, poznatky, zkušenosti, výsledky pozorování
- **informace** – smysluplná interpretace dat
- **databáze** – integrovaná počítačově zpracovaná množina persistentních dat
- **relační model dat** – založený na predikátové logice prvního řádu; k manipulaci s daty lze použít relační kalkuly nebo operace relační algebry
- **dotazovací jazyk** – slouží pro získání dat z databáze; v relační databázi je odpovědí relace, jejíž n-tice splňují nadefinované podmínky
- **systém řízeníází dat (SŘBD / DBMS)** – množina programových prostředků, která umožňuje: vytvoření databáze, použití databáze (manipulace s daty), údržbu a správu databáze
- **databázový systém (DBS)** – SŘBD + databáze

Dělení databází podle typu

Databáze se dají dělit podle typu a způsobu ukládání dat na tři hlavní kategorie:

1. Relacíní databáze

Nejčastěji používaný typ databází. Data jsou modelována pomocí relačního modelu, tedy tabulek s fixní strukturou jednotlivých záznamů, které jsou navzájem propojené pomocí relací. V tomto typu databáze se používá relační model, což je matematický koncept ze 70. let, tvořený množinami (tabulkami), přičemž každá tabulka má svůj primární klíč (jeden nebo více atributů – composite primary key). Příklady: Oracle, MySQL, PostgreSQL, MS SQL Server.

2. NoSQL databáze

Nemají tabulky, jsou nestrukturované – rychlejší, dávají všechna data dohromady.

- **Výhody:** rychlost, škálovatelnost
- **Nevýhody:** NoSQL databáze nepoužívají tabulkové struktury, normalizaci a SQL; namísto toho jsou tvořeny tzv. kolekcemi, ve kterých jsou uložené dokumenty. Dokument je většinou komplexní objekt tvořený mnoha dílčími podobjekty. Nejpoužívanější NoSQL databáze: MongoDB, CouchDB, Amazon DynamoDB.

Uložení dat do JSON nebo jeho efektivnější binární verze BSON.

3. Specializované databáze

Poslední, velice obecnou kategorií jsou specializované databáze. Jedná se často o databáze, které řeší nějaký konkrétní problém a nehodí se pro obecné use cases jako RDBMS nebo NoSQL.

- **Graph Databases** – databáze nativně podporující grafové struktury pro modelování komplexních sítí (např. Neo4J)
- **Key-Value databáze** – ukládají vždy jednu hodnotu pro daný klíč (např. Redis, Memcached); Key-Value se většinou používá jako cache nad „tradiční“ databází
- **Time-series databáze** – optimalizované pro časové řady dat
- **Sloupcové databáze** – data ukládána po sloupcích (Cassandra, HBase)

1.1 Analýza, návrh a realizace databáze

1.1.1 Databázový systém a jeho vlastnosti

Databázový systém (DBS) = SŘBD + Databáze

Důvody pro vznik databázových systémů:

- **Vyšší datová abstrakce** – SŘDB zahrnují manipulační jazyky pro práci s datovými strukturami vyšší úrovně
- **Nezávislost aplikačních programů** na změnách ve fyzickém uložení dat – při změnách ve fyzické struktuře báze dat není třeba měnit aplikační programy, protože se na data obrazejí na logické úrovni
- **Ochrana dat** před neoprávněným přístupem a poruchami
- **Odstranění redundance** – každý údaj je ve většině případů uložen pouze jedenkrát
- **Sdílení dat** – při využití SŘDB mohou být taktéž data používána několika uživateli
- **Podpora transakčního zpracování**
- **Údržba integrity databáze**

1.1.2 Metodologie analýzy a návrhu databáze – P3A

Návrh databáze vychází z principu tří architektur (**P3A**). Smyslem P3A je postupně upřesňovat datový model tak, aby zcela odpovídal požadavkům využívané databázové technologie. Stejně tak je naopak žádoucí, aby model v první vrstvě byl, pokud možno, abstraktní a technologicky nezávislý.

1. Konceptuální model (model reality) Model obsahu datové základny na konceptuální úrovni (tj. nezatížený jakýmikoliv implementačními a technologickými implementacemi). Konceptuální schéma představuje přenesení reality do databáze. Neřeší implementační detaily, ale modeluje pouze entity a vazby mezi nimi.

- **KSR** (Konceptuální schéma reality) – hrubý model obsahující základní entitní množiny, vztahy a jejich atributy. Jedná se o hrubý model vytvořený za účelem poznání zkoumané reality.
- **KSD** (Konceptuální schéma dat / datový model) – popis obsahu datové základny za účelem přesné specifikace obsahu datové základny. Předběžný návrh datové základny.

Funkce konceptuálního schématu:

- Zadání pro implementační fázi
- Nástroj (záruka) pro zachování konsistence
- Lze z něj odvozovat dílčí pohledy na obsah databáze
- Východisko pro nové požadavky na data
- Východisko pro zásahy do existující datové základny
- Nástroj dokumentace

2. Logický (technologický) model Popis způsobu realizace systému v termínech jisté třídy technologického prostředí (lineární, relační, hierarchické, síťové datové struktury). Formální matematický model reprezentující konceptuálního schématu za pomoci tabulek a jejich relací. Každá tabulka je vyjádřena pomocí schématu, který udává názvy a datové typy sloupců v ní.

Pravidla transformace do relačního modelu:

- Každá entitní množina je transformována do jedné relační tabulky; identifikátory entitní množiny se stanou atributy tvořícími primární klíč relační tabulky
- Vztah z konceptuálního modelu je v relačním modelu vyjádřen cizím klíčem
- Vztah M:N z konceptuálního modelu je v relačním modelu vyjádřen vazební relační tabulkou a dvěma cizími klíči
- Generalizace/specializace je transformována do relačního modelu dat několika variantami:
 - jednou relační tabulkou na úrovni celé hierarchie
 - relačními tabulkami na úrovni specializovaných entitních množin
 - relačními tabulkami na úrovni generalizující entitní množiny i na úrovni specializovaných entitních množin
- Využívá se taktéž metoda normalizace dat

Cíle logického modelu:

- Vytvořit co nejvěrnější obraz v modelovaném světě existujících entitních množin
- Zajistit interní konzistenci datového modelu, resp. databáze
- Minimalizovat redundanci
- Maximalizovat stabilitu datových struktur

- Sada normálních forem (1NF, 2NF, 3NF, BCNF, 4NF, 5NF)

3. Fyzický (implementační) model Popis vlastní realizace systému v konkrétním implementačním prostředí (doplnění i typu indexu, velikosti, rozmístění pracovního prostoru apod.). Je dán:

- **Způsobem uložení dat:**
 - *Sekvenční* – způsob uložení dat není využíván pro uložení dat v databázových systémech; bývá používán pro zálohování databázových souborů
 - *Přímé* – hodnotě primárního klíče jednotlivých záznamů je přiřazena (vypočtena) určitým algoritmem (hashing algorithm) adresa uložení záznamu na paměťovém médiu
- **Způsobem přístupu k datům:** sekvenční, přímé, s využitím dalších mechanismů (řetězení, indexy)

1.1.3 Návrh databáze

Návrh databáze spočívá v určení entit (tabulek), klíčů pro zajištění referenční integrity, vztahů neboli relací mezi tabulkami, dodržení normálních forem a definice indexů pro lepší výkon.

Postup konceptuálního návrhu databáze Cílem je vytvořit ER (Entity Relationship) model datových požadavků organizace (nebo části organizace), kterou má databáze podporovat. Každý ER model obsahuje:

- Entity
- Relace
- Atributy a domény atributů
- Primární klíče a alternativní klíče
- Integritní omezení

Jednotlivé kroky konceptuálního návrhu:

1. **Identifikace entit** – identifikace a dokumentace hlavních entit v organizaci
2. **Identifikace relací** – identifikace důležitých relací mezi entitami, určení omezení multiplicity těchto relací
3. **Identifikace atributů a jejich spojení s entitami nebo relacemi** – spojení atributů s odpovídajícími entitami nebo relacemi; identifikace jednoduchých/složených atributů, atributů s jedinou hodnotou/atributů s více hodnotami a odvozených atributů
4. **Určení domén atributů** – domén (množina hodnot, z níž mohou čerpat své hodnoty jeden nebo více atributů) pro atributy v ER modelu
5. **Určení kandidátních, primárních a alternativních klíčů**

6. **Specializace/generalizace entit (volitelný krok)** – určení entit nadtříd a podtříd
7. **Kontrola redundance v modelu** – kontrola ER modelu, zejména kontrola relací typu 1:1 a odstranění redundantních relací
8. **Kontrola, zda model podporuje uživatelské transakce**
9. **Posouzení konceptuálního návrhu databáze s uživateli**

1.1.4 Normalizace databáze

Normalizace je proces organizace dat v databázi tak, aby se minimalizovala redundance a zajistila konzistence dat. Normalizace se řídí sadou pravidel – normálních forem.

1. normální forma (1NF) Relace je v 1NF, pokud:

- Každý atribut obsahuje pouze atomické (nedělitelné) hodnoty
- Neexistují opakující se skupiny atributů
- Každý řádek je jednoznačně identifikovatelný (existuje primární klíč)

2. normální forma (2NF) Relace je v 2NF, pokud:

- Je v 1NF
- Každý neklíčový atribut je plně funkčně závislý na celém primárním klíči (nikoliv pouze na jeho části) – eliminuje částečné závislosti

2NF je relevantní pouze tehdy, pokud má tabulka složený primární klíč.

3. normální forma (3NF) Relace je v 3NF, pokud:

- Je v 2NF
- Neexistují tranzitivní závislosti neklíčových atributů na primárním klíči (žádný neklíčový atribut nesmí být funkčně závislý na jiném neklíčovém atributu) – eliminuje tranzitivní závislosti

Boyce-Coddova normální forma (BCNF) Relace je v BCNF, pokud:

- Je v 3NF
- Pro každou netriviální funkční závislost $X \rightarrow Y$ platí, že X je nadmnožinou kandidátního klíče (suprakilče) – silnější verze 3NF

Denormalizace Proces porušování pravidel daných normalizací dat (vzdalování se od modelu světa v databázi) s cílem zvýšení výkonnosti systému:

- **Výhody:** snižování potřeby spojování tabulek, uchování aktuálních hodnot odvozených atributů
- **Nevýhody:** snížení rychlosti aktualizacích operací (I, U, D), složitější aplikační řešení, obtížnější zajištění integrity databáze

1.1.5 Integrita a transakce

Integritní omezení Pravidla pro zajištění správnosti a konzistence uložených dat:

- **Entitní integrita** – výskyt každé entity musí být jednoznačně identifikovatelný; musí existovat primární klíč (UNIQUE, NOT NULL)
- **Referenční integrita** – cizí klíč obsahuje pouze hodnoty, které existují v jemu odpovídajícím primárním klíči; odkazování pouze na existující záznamy v povolené kardinalitě
- **Doménová integrita** – atribut může obsahovat pouze přípustné hodnoty omezené integritním pravidlem (CHECK, TRIGGER); stanovení povolených hodnot ve sloupci

Transakce a vlastnosti ACID Transakce slouží k zajištění konzistence a řízení více-uživatelského přístupu. Vlastnosti transakce – **ACID**:

- **A (Atomicity)** – transakce je jako celek, musí proběhnout celá nebo žádná
- **C (Consistency)** – transformuje DB z jednoho konzistentního stavu do jiného konzistentního
- **I (Isolation)** – transakce jsou nezávislé, změny prováděné jsou neviditelné pro ostatní transakce
- **D (Durability)** – efekty potvrzené transakce jsou uloženy do databáze
- **OLAP** (Online Analytical Processing) – zpracování vícedimenzionálních dotazů (datové sklady); zaměřeno na analytické dotazy nad velkým množstvím historických dat
- **OLTP** (Online Transaction Processing) – zpracování velkého množství relativně malých transakcí; zaměřeno na každodenní operace (INSERT, UPDATE, DELETE)

1.1.6 Indexy

Indexy jsou databázové objekty sloužící ke zrychlení vyhledávacích a dotazovacích procesů v databázi, definování unikátní hodnoty sloupce tabulky nebo optimalizaci fulltextového vyhledávání.

- Každý index zabírá v paměti vyhrazené pro databázi nezanedbatelné množství místa

- Každý index zpomaluje operace, které mění obsah indexovaných sloupců (INSERT, UPDATE) – databáze se v případě takové operace nad indexovaným sloupцем musí postarat nejen o změny v datech tabulky, ale i o změny v datech indexu
- Pro správné zvolení indexů by ten, kdo databázi navrhuje, měl vědět, jak často se vybírané záznamy z dané tabulky třídí podle zamýšleného sloupce (kandidáta na index) a nakolik je důležité, aby vyhledávání a třídění podle něj bylo rychlé

Otázky profesorů k tématu: Analýza, návrh a realizace databáze

Komise: Svátek, Chlapek, Zbořil

- Jaký by měl být správný postup analýzy, návrhu a realizace databáze?

Máša Petr, RNDr. Ing., Ph.D.

- Jaké budou problémy aplikace vysoce náročné na data, jak řešit v tomto případě řešení databáze?
- Co se rozumí principem 3NF?
- Jaké aplikace by se daly doporučit pro realizaci dané aplikace?

Sládek Pavel, Ing., Ph.D.

- Jaké jsou druhy databází a jak fungují?

Chudán David, Ing., Ph.D.

- Jak se typicky strukturovaná data ukládají? Jaké znáte typy databází?
- Jaké jsou další možnosti pro ukládání strukturovaných dat? Jaké jsou rozdíly mezi formáty CSV, JSON a XML?

Zeman Václav, Ing., Ph.D.

- Jak databáze zajistí konzistenci a integritu řešení?

Strossa Petr, doc. RNDr., CSc.

- V čem se liší NoSQL od relačních databází?
- Jaký je vyhledávací interface pro relační databáze?
- K čemu slouží indexy v databázi?

Komise: Strossa, Sedláček, Kučera

- Jaký je vztah mezi pojmy entitní množina a relační tabulka?

Sudzina František, Mgr. et Mgr. Ing., Ph.D.

- Jaký druh databází byste použil pro ukládání informací předcházejících dopravní nehodě?
- Jaké jsou výhody databází v porovnání s tabulkami např. v Excelu?

Komise: Sklenák, Vojtěch, Palovský

- Jaké jsou typy databází? Příklady relačních databází.

Maryška Miloš, doc. Ing., Ph.D.

- Co je normalizace a denormalizace dat?

1.2 Datové modelování

1.2.1 Základní pojmy

- **Entita** – typ objektů, ať už konkrétních (osoby, místa, věci) či abstraktních (katalogové položky, kategorie), natolik důležitých v dané realitě, že se o takových objektech mají vést záznamy; tabulka (ale až v DB), která vykazuje stejné společné znaky. V ER modelech modelujeme entitní typy (např. entitní typ: STUDENT, entita: Ondřej

Horák).

- **Objekt** – jiné označení pro entitu
- **Atribut** – vlastnost nebo charakteristika objektu nebo vztahu v datovém modelu (např. jméno a příjmení zaměstnance). Typy atributů/sloupce:
 - *Povinný* – musí mít hodnotu (NOT NULL)
 - *Volitelný* – může nemít hodnotu (NULL)
 - *Vícehodnotový* – má hodnoty definované v dané doméně
 - *Skupinový / složený* – vnitřní struktura, např. u adresy (ulice, město, PSČ) – obvykle se normalizuje
 - *Atomický* – není vícehodnotový ani skupinový
- **Vztah (relace)** – pojmenovaná spojitost mezi entitami; mezi entitami lze popsat vztah větou, ve které vztah vyjádříme přísudkovou částí (např. „Zákazník vytváří Objednávku“)
- **Kardinalita** – četnost vazby; nabývá hodnot 1:1, 1:N, M:N. Říká, kolik výskytů entitního typu jednoho může být v daném vztahu s jedinou entitou druhého typu.
- **Parcialita** – volitelnost vztahu mezi entitami; říká, zda všechny výskyty entitního typu, jenž je určen pro danou roli v tomto vztahu, musí do tohoto vztahu skutečně vstupovat
- **Primární klíč (PK)** – atribut nebo kombinace atributů jednoznačně identifikující každý záznam v tabulce (NOT NULL, UNIQUE)
- **Cizí klíč (FK)** – atribut nebo kombinace atributů odkazující na primární klíč jiné tabulky; zajišťuje referenční integritu

1.2.2 ER (Entity Relationship) modelování

ER (Entity Relationship) modelování se používá pro abstraktní a konceptuální znázornění dat. Spočívá ve využití základních konstruktů jazyka pro tvorbu diagramů a v metodice tvorby těchto diagramů – základní myšlenkou je, že databáze uchovává fakta o entitách a o vztazích mezi entitami. Výsledkem je **ERD** (Entity Relationship Diagram, ER diagram).

Zásady ER modelování

- Minimalizace redundance (normalizace)
- Maximalizovat znovupoužitelnost (dědičnost)
- Maximalizovat výkonnost (de-normalizace, metody, database tuning)
- Minimalizovat nároky na uložení dat (compression)

1.2.3 Datové modelování – činnosti

1. Rozlišení množin objektů (entitních množin)
2. Pojmenování entitních množin a identifikace entit
3. Rozlišení entitních podmnožin (zaměstnanec – učitel, vědec, administrátor, zaměstnanec)
4. Určení vztahů mezi entitními množinami, určení kardinality (počet) a parciality (volitelnost) vztahů
5. Určení atributů entitních množin a vztahů
6. Vyřešení problémů synonym a homonym

1.2.4 Transformace do relačního modelu dat

Data v tabulce jsou uložena v polích uspořádaných do řádků (= záznamy) a sloupců (= atributy).

Konceptuální prvek	Relační prvek
Entity	Tabulky
Atributy	Sloupce
1:N vztahy	Cizí klíč (FK)
M:N vztahy	Asociativní tabulky (vazební tabulka se 2 FK)
Identifikátory	Primární klíče (PK)
Volitelnost	Povolené hodnoty NULL pro FK
Povinnost	NOT NULL pro FK

Zpracování vztahů

- **1:1** – záznam obsahuje sloupec pro zapsání právě jednoho primárního klíče jiného záznamu
- **1:M** – záznam obsahuje sloupec pro zapsání primárního klíče jiného záznamu; na onen jiný záznam se může odkazovat více různých záznamů
- **M:N** – může existovat více vzájemných odkazů; realizováno pomocí samostatné vazební tabulky (dekompozice, neboli rozdělení na dvojici vztahů 1:M)

1.2.5 Modely databázových systémů

Hierarchický (stromový) model dat Data jsou organizována do stromové struktury. Každý záznam představuje uzel ve stromové struktuře, vzájemný vztah mezi záznamy je typu rodič/potomek. Nalezení dat v hierarchické databázi vyžaduje navigaci přes záznamy směrem na potomka, zpět na rodiče nebo do strany na dalšího potomka. Největšími nevýhodami hierarchického uspořádání je složitá operace vkládání a rušení záznamů.

Síťový model dat Síťový model dat je v podstatě zobecněním hierarchického modelu, který doplňuje o mnohonásobné vztahy (sety). Tyto sety propojují záznamy různého či stejného typu, přičemž spojení může být realizováno na jeden nebo více záznamů.

Relační model dat V roce 1970 byl popsán Dr. Coddem. V současnosti je nejčastěji využíván v komerčních SŘBD. Model má jednoduchou strukturu – data jsou organizována v tabulkách, které se skládají z řádků a sloupců. V těchto tabulkách jsou prováděny všechny databázové operace.

Databáze dle relačního modelu musí splňovat:

- Databáze je chápána uživatelem jako množina relací a nic jiného
- V relačním SŘBD jsou k dispozici minimálně operace selekce, projekce a spojení, aniž by se vyžadovaly explicitně předdefinované přístupové cesty pro realizaci těchto operací

Objektově-orientované DBS (OODBS) Data v OODBS jsou chápána jako objekty odpovídající přímo entitám reálného světa. Objekty zahrnují data i chování, tj. funkčnost ve stávajících RDBS zajištěnou aplikačními programy.

- Komplexní objekty (kromě relačních tabulek i další typy objektů)
- Identita objektů (DBS zajišťuje unikání identifikaci objektu v rámci celé DB – OID)
- Logická datová nezávislost (zapouzdření, skrývání dat + zveřejnění rozhraní)
- Dědičnost (odvozování nových tříd z existujících)
- Polymorfismus (schopnost operací fungovat na objektech více než jednoho typu)

1.2.6 Trendy v databázových systémech

DaaS (Database as a Service)

- Služba by měla být k dispozici zákazníkovi on-demand (na vyžádání), bez nutnosti instalovat a konfigurovat hardware nebo software
- Poskytovatel je odpovědný za správu služby, zákazník nemusí systém upgradovat nebo spravovat
- **Výhody:** škálovatelnost, snadné používání, nižší náklady na infrastrukturu
- **Nevýhody:** bezpečnostní rizika, závislost na poskytovateli, nutnost spolehlivého internetového připojení

Další trendy

- Přechod na NoSQL pro nestrukturovaná data
- Databáze ve vnitřní paměti (in-memory databases) – radikálně mění vyhodnocování dotazů

- Paralelní zpracování, distribuované databáze
- Federace milionů databázových systémů
- Integrace strukturovaných a semistrukturovaných dat (XML)
- Datové sklady (pumpování dat z OLTP databází, jejich čištění a uložení do speciálních – OLAP databází)

Otázky profesorů k tématu: Datové modelování

Řepa Václav, prof. Ing., CSc.

- Najděte ve Vašem “relačním” modelu potenciální nedostatky (co nežádoucího model připouští). Jaké jsou možné přístupy k eliminaci takových nedostatků?
- Jak jsou realizovány vztahy mezi entitami v relačních databázích?
- Jaký je rozdíl mezi datovým a procesním modelem?
- Jaké architektury návrhu datového modelu znáte (princip 3 architektur)? Jaké typy modelů byste vám mohly pomoci?
- Jaké znáte normální formy? Jsou jádro datového skladu a data marty normalizované a denormalizované?

Kučera Jan, Ing., Ph.D.

- Jak je možné při vývoji aplikace, jako je ta, kterou jste vytvářel v rámci bakalářské práce, využít modelování s využitím jazyka UML?
- Znáte grafický prostředek, který se specificky používá pro konceptuální modelování?

Pour Jan, doc. Ing., CSc.

- Jaké jsou principy a problémy datového modelování?
- Co je principem datových modelů?
- Modelování podnikových systémů, jaké jsou hlavní principy?
- Kde jsou klíčové problémy s modelováním podnikových procesů?

Maryška Miloš, doc. Ing., Ph.D.

- Datové modelování a jeho vrstvy.
- Co je normalizace a denormalizace dat?

Komise: Strossa, Sedláček, Kučera

- Jaký je vztah mezi pojmy entitní množina a relační tabulka?

1.3 Databázové jazyky

1.3.1 Dělení databázových jazyků

Databázový jazyk je prostředek komunikace člověka s databázovým systémem.

Pravidlo komplexního datového jazyka: Relační systémy mohou podporovat více jazyků a režimů přístupů, ale musí existovat minimálně jeden jazyk, jehož příkazy jsou vyjádřitelné dobře definovanou syntaxí jako řetězce znaků, který podporuje definici dat, manipulaci s daty, ochranu dat a omezení integrity (lze považovat za standard).

Základní dělení jazyků podle nejčastěji identifikovaných částí:

- **DDL (Data Definition Language)** – definice databázových objektů; příkazy CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – manipulace s daty; příkazy SELECT, INSERT, UPDATE, DELETE
- **DCL (Data Control Language)** – řízení přístupu k datům; příkazy GRANT, REVOKE
- **TCL (Transaction Control Language)** – řízení transakcí; příkazy COMMIT, ROLLBACK, SAVEPOINT

1.3.2 Databázové jazyky 4. generace (4GL)

Databázové jazyky 4. generace (4GL: 4th Generation Language) jsou pokročilé programovací jazyky, které dovolí laikům (neprogramátorům) psát krátké programy extrahující data z databází a vytvářet reporty:

- FOCUS, Informix-4GL, Quel, Progress 4GL
- SQL (v současné době nejrozšířenější)

1.3.3 SQL (Structured Query Language)

Standardizovaný dotazovací jazyk používaný pro práci s daty v relačních databázích. SQL je zkratka anglických slov **Structured Query Language** (strukturovaný dotazovací jazyk).

V 70. letech 20. století probíhal ve firmě IBM výzkum relačních databází. Vznikl tak jazyk **SEQUEL** (Structured English Query Language). Relační databáze byly stále významnější a bylo nutné jejich jazyk standardizovat. Standardizován postupně organizacemi ISO, IEC a ANSI.

Vývoj standardů SQL

Rok	Označení	Charakteristika
1986	SQL-86	První verze (ANSI/ISO 1987)
1989	SQL-89	Oprava a mírné rozšíření normy
1992	SQL-92	Označení SQL2, komplexní pohled, tři úrovně
1999	SQL:1999	SQL3 – regulární výrazy, rekurzivní dotazy, trigger, OO vlastnosti
2003	SQL:2003	Podpora XML, GUI funkce, multimédia
2006	SQL:2006	Import/export XML dat, vazba na XML Query Language
2008	SQL:2008	Kompletní přepracování všech hlavních částí

Příklady použití SQL

Listing 1: Vytvoření tabulky

```
CREATE TABLE zamestnanci (
    ID_zam INT PRIMARY KEY,
```

```

    jmeno      VARCHAR(50) NOT NULL,
    prijmeni   VARCHAR(50) NOT NULL,
    plat       DECIMAL(10,2)
);

```

Listing 2: Vložení dat

```

INSERT INTO zamestnanci (ID_zam, jmeno, prijmeni, plat)
VALUES (1, 'Jan', 'Novák', 45000.00);

```

Listing 3: Výběr dat

```

-- Všechny záznamy
SELECT * FROM zamestnanci;

-- Podmíněný výběr
SELECT jmeno, prijmeni FROM zamestnanci
WHERE plat > 40000
ORDER BY prijmeni;

-- Spojení tabulek (JOIN)
SELECT z.jmeno, z.prijmeni, o.nazev AS oddeleni
FROM zamestnanci z
JOIN oddeleni o ON z.oddeleni_id = o.id;

```

Listing 4: Aktualizace a mazání dat

```

-- Aktualizace
UPDATE zamestnanci
SET plat = 50000.00
WHERE ID_zam = 1;

-- Mazání
DELETE FROM zamestnanci
WHERE ID_zam = 1;

```

Listing 5: Transakce

```

BEGIN TRANSACTION;
    UPDATE ucty SET zustatek = zustatek - 1000 WHERE id = 1;
    UPDATE ucty SET zustatek = zustatek + 1000 WHERE id = 2;
COMMIT;
-- nebo ROLLBACK; pro zrušení transakce

```

1.3.4 PL/SQL

Procedural Language / Structured Query Language je procedurální nadstavba jazyka SQL od firmy Oracle, založená na programovacím jazyku Ada.

Pomocí PL/SQL je možno vytvářet:

- **Uložené procedury a funkce** – předkompilované programové bloky uložené v databázi
- **Programové balíky** – skupiny příbuzných procedur a funkcí
- **Triggery** – automaticky spouštěné kódy při určitých databázových událostech (INSERT, UPDATE, DELETE)
- **Uživatelsky definované datové typy**

Tato nadstavba se rozšířila a její deriváty převzaly i jiné relační databáze.

1.3.5 OQL (Object Query Language)

Object Query Language (OQL) je standardizovaný dotazovací jazyk pro objektově-orientované databáze vytvořený podle SQL. OQL byl vyvinut Object Data Management Group (ODMG). Kvůli jeho komplexitě zatím žádný výrobce neimplementoval OQL v úplnosti.

OQL je záměrně navržen tak, aby byl velmi podobný jazyku SQL-92. Část syntaxe SQL, která se týká manipulace s daty, je kompletně obsažena v OQL. OQL nezbyvá tvorbou datových struktur – příkazy SQL týkající se definice dat (create-table, create-index atd.) nemají v OQL žádný ekvivalent.

1.3.6 JPQL (Java Persistence Query Language)

Java Persistence Query Language je platformě nezávislý objektově-orientovaný dotazovací jazyk definovaný jako součást specifikace Java Persistence API (JPA). JPQL se používá k dotazování se na entity uložené v relační databázi. Je silně inspirován SQL a má podobnou syntaxi.

JPQL odštiňuje programátora od specifik konkrétního datového úložiště – programátor se nestará o to, zda jsou v konečném důsledku dotazy prováděny nad Oracle nebo MySQL databází.

1.3.7 XQuery

XQuery je dotazovací a funkcionální programovací jazyk navržený pro dotazování na kolekce dat v XML. XQuery poskytuje prostředky pro extrahování a manipulaci dat z XML dokumentů nebo jakéhokoli datového zdroje, který může být jako XML zobrazen (např. relační databáze nebo některé dokumenty jako OpenOffice, MS Office 2007).

XQuery používá **syntax jazyka XPath** pro adresování konkrétních částí XML dokumentů. To je doplněno SQL-podobnými „**FLWOR**“ **výrazy** pro provádění joinů. FLWOR výraz je sestaven z pěti klauzulí: FOR, LET, WHERE, ORDER BY, RETURN.

1.3.8 SQL injection – bezpečnostní hrozba

SQL injection je jedna z nejčastějších útoků na databáze přes webové skriptovací jazyky. Útočník vloží škodlivý SQL kód do vstupního pole aplikace, který se pak vykoná v data-

bázi.

Příklad zranitelného kódu

```
-- Zranitelný dotaz (vstup od uživatele: ' OR '1'='1')
SELECT * FROM users
WHERE username = '' OR '1'='1' AND password = '';
-- Vráťí všechny záznamy!
```

Ochrana proti SQL injection

- **Prepared statements (parametrizované dotazy)** – oddělení SQL kódu od dat
- **ORM frameworky** – objektově-relační mapování eliminuje přímé SQL dotazy
- **Validace vstupů** – kontrola a sanitace uživatelských vstupů
- **Nejmenší oprávnění** – databázový uživatel má pouze nutná oprávnění
- **WAF (Web Application Firewall)** – filtrování škodlivých požadavků

Otázky profesorů k tématu: Databázové jazyky

Strossa Petr, doc. RNDr., CSc.

- V čem se liší NoSQL od relačních databází?
- Jaký je vyhledávací interface pro relační databáze?
- K čemu slouží indexy v databázi?

Sedláček Jiří, Ing., Ph.D.

- Jaké typy útoků jsou časté při dotazování databází přes webové skriptovací jazyky? Jak se jim bránit?

Komise: Vojíš, Pour, Sudzina

- Jaké jsou výhody a nevýhody NoSQL databází?

Zeman Václav, Ing., Ph.D.

- Jak databáze zajistí konzistenci a integritu řešení?

Maryška Miloš, doc. Ing., Ph.D.

- Popište architekturu datově orientovaného řešení.